Tutorial 9

Quiz 2: Pots of Gold

Problem. Let $G[0 \ldots (n-1)]$ be an array of n positive integers. In the *pots-of-gold game*, there are n pots $P_0, P_1, \ldots, P_{n-1}$ lined up in a row, and pot P_i contains $G[i]$ gold coins. Two players start with an empty bag each, and they take turns removing *exactly one pot from either end* of the remaining row. Each player adds the gold coins from the removed pot to their own bag. The game ends when no pots remain.

Our goal: find the **maximum** total gold that the **first player** can **guarantee** to collect, no matter how the second player responds.

Quiz 2: Examples

Array G	n	**Correct output**
$[5]$	1	5
$[10]$	1	10
$[1, 100]$	2	100
$[3, 9]$	2	9
$[3, 2, 7]$	3	9
$[1, 2, 3]$	3	4
$[10, 1, 10]$	3	11

DP Solution 1: 2D Table ($n \times n$)

DP Solution 2: 3D Table ($n \times n \times 2$)

Wine Barrels

I had almost included this in the handout... But then decided against it. I am not sure how to feel about that.

Problem. There are N barrels of wine stored in a narrow passage. Each year, you sell either the *first* or the *last* barrel.

The i-th wine has initial price $P[i]$, and sells for $k \cdot P[i]$ in the k-th year.

Goal: What is the *maximum possible total profit*?

Player 1 Wins? (P19, Problem Set 1)

But then again, Prof. Philip had already asked it in Problem Set 1 (and it was discussed in Tutorial 5) **Problem.** You are given an integer array

`nums`. Two players take turns, with player 1 starting first. Both players start with score 0.

At each turn, the player takes one of the numbers from either end of the array (`nums[0]` or `nums[nums.length - 1]`), reducing the array size by 1. The player adds the chosen number to their score.

The game ends when no elements remain.

Return `true` if Player 1 can win. If scores are equal, Player 1 is still considered the winner. Assume both players play optimally.

Calvin and Hobbes (Induction & Recursion, Week 2)

Problem. Calvin and Hobbes wish to divide 25 coins, of denominations $1, 2, 3, \ldots, 25$ kopeks.

In each move, one of them *chooses* a coin, and the *other player decides* who must take it.

Rules:

- Calvin makes the initial choice.
- In subsequent moves, the choice is made by the player currently having *more* kopeks.
- In case of a tie, the same player who chose in the preceding move chooses again.

After all coins are taken, the player with more kopeks wins.

Which player has a winning strategy?

Solution

Can we try to encode everything about the game in a single state? For example a chess board state and whose turn tells us everything we want to know, can we do the same here?
As it turns out, yes. That would be $([1, 2, \ldots, 25], 0, 0,' J')$. Let's call a positions N if the picking player looses from there via some move and P if the picking player wins irrespective of the moves.

Note that there is indeed a winner because $1 + dots + 25 = 325$ is odd.

We claim that Hobbes wins. For every initial coin Calvin may choose, Hobbes can decide to either take it or give it to Calvin. These two scenarios are exactly mirror opposites (aka same upto relabbeling of players), so Hobbes wins in exactly one of them, and he chooses precisely that one.

Greedy Ideas in DP: $O(n)$ Time and $O(1)$ Space Solution

Problem. For the pots-of-gold / coin row games, is there an $O(n)$ time and $O(1)$ space solution?

Let the *val* be the difference between scores of player 1 and player 2. Now observe:

Greedy: If the maximum element M is at one of the ends of the array A then, $val(A) = M - val(A')$ where $A' = A.remove(M)$
Replacement: If there is a pattern (all adjacent to each other) xMy in A such that $x, y < M$ then $val(A) = val(A')$ in A' where xMy is replaced by $x + y - M$. Note M is not the maximum element here.

We can use the replacement idea to convert the array to an array that is first decreasing then increasing; and solve that using greedy as it always has the maximum element at ones of the end.

Greedy Ideas in DP: Chocolate Bar Breaking

Problem. We are given a bar of chocolate composed of $m \times n$ square pieces. The chocolate must be broken into individual single squares.

Parts may be broken along vertical and horizontal lines. A **single break** divides one piece into two smaller pieces.

Each break incurs a **cost** (a positive integer), depending only on the line chosen, not on the size of the piece.

The **total cost** is the sum of costs of all breaks.

Compute the minimum possible total cost required to break the entire chocolate bar into single square
(i) Using DP
(ii) Using Greedy (no need to prove this yet!)

Ulta DP: Longest Increasing Subsequence (LIS)

Problem. Given an array $A[1 \ldots n]$, find the length of the longest strictly increasing subsequence.

Consider a recursion which given an integer from 1 to n, gives the smallest element at which an increasing subsequence of length l ends. Can you see an $O(n \log(n))$ algorithm?

Ulta DP: Ace and Blackbeard (Pre-Mid P6)

Problem. Ace has been captured by Blackbeard and is held prisoner aboard the Saber of Xebec, a massive ship with n cabins. Blackbeard is demanding a ransom.

Nami, negotiating on behalf of the Straw Hats, agreed to the following terms. She will guess the cabin number where Ace is held.

- If she guesses **correctly**, Blackbeard releases Ace.
- If her guess is **too high**, she pays a Berries.
- If her guess is **too low**, she pays b Berries.

We want to compute the minimum cost Nami can incur using the optimal strategy, irrespective of where Ace is imprisoned. Can you use an 'Ulta' DP on this? Where $dp[l]$ reprasents the maximum sized ship we can search at cost of l berries.

Matter / Antimatter

Problem. You are given n devices in a line, each with value $A[i]$. For any **contiguous subarray**, you may assign each device either $+A[i]$ (matter) or $-A[i]$ (antimatter).

A selection is **valid** if the total sum equals 0 (matter equals antimatter).

Count the **number of different ways** to:

- choose a contiguous subarray, and
- assign $+$ or $-$ signs to its elements

such that the total sum is 0.

Two ways are different if the subarray differs *or* at least one sign differs.

Example: For $A = [1, 1, 1, 1]$, the answer is **12**.

Gaius Julius Caesar's Soldiers

Problem. An army has n_1 footmen and n_2 horsemen. An arrangement is **not beautiful** if:

- there are strictly more than k_1 footmen standing successively, or
- there are strictly more than k_2 horsemen standing successively.

Find the **number of beautiful arrangements** of all $n_1 + n_2$ soldiers.

Notes:

- All $n_1 + n_2$ soldiers must appear in each arrangement.
- All footmen are indistinguishable among themselves; similarly for horsemen.

Packing Problem

Problem. You have n suitcases, each with capacity $S[i]$.

You have m goods with weight $W[i]$ and value $V[i]$.

Using all the suitcases, what is the **maximum value** one can carry?

(How does this differ from the classical knapsack problem?)

We expect a DP which would take $O(m^2 n \prod_{i=0}^{n-1} S[i])$ time.

Divide & Conquer: Counting Inversions

Problem. Given a (1-indexed) array A of n distinct integers, give an algorithm to count the number of inversions in $O(n \log n)$.

An **inversion** is a pair (i, j) such that:

- $1 \leq i < j \leq n$, and
- $A[i] > A[j]$.

Hint: Consider a function

```
def inversions(A: list[int]) -> tuple[int, list[int]]
```

where we return both the *number of inversions* and the *sorted* list A.

Divide & Conquer: Merging k Sorted Arrays

Problem. Let $A_1, A_2, \ldots, A_k$ be k arrays, each of which has been sorted.

These arrays are **mutually disjoint**: no integer can appear in more than one array.

Design an algorithm to merge the k arrays into one sorted array in $O(n \log k)$ time, where n is the total length of the k arrays.

Note: These arrays may have different lengths.

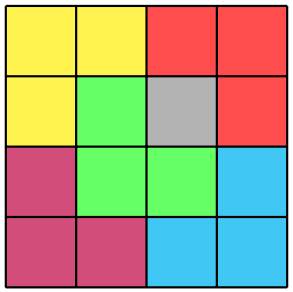
Divide & Conquer: L-Tiles

Problem. Given a grid of $2^N \times 2^N$, all cells are initially empty except one cell B.

Write an algorithm to fill the grid with L-shaped trominoes (without overlapping other tiles or B), or report "NO" if not possible. Your algorithm should run in $O(2^N)$.

Hint: We never have to return "NO".

The $N = 2$, $B = (3, 2)$ case:



Divide & Conquer: Big Mod

Problem. Given p, e, b, compute $p^e \bmod b$ in $O(\log e)$ time.

Cake Cutting

Problem. There is a cake represented by the continuous interval $[0, 1]$. There are n people such that, given $0 \leq l < r \leq 1$, each person i can report their valuation $f_i(l, r) \in [0, 1]$ for that piece, with $f_i(0, 1) = 1$ always.

Divide the cake into $[l_1, r_1], [l_2, r_2], \ldots, [l_n, r_n]$ such that:

$$f_i(l_i, r_i) \geq f_i(l_j, r_j) \quad \text{for all } i, j$$

That is, every person i values their own piece at least as much as anyone else's piece.
Also, $l_1 = 0, r_n = 1$ and $r_i = l_{i+1}$
Calling any of the n functions takes $O(1)$ time.

Solution

I wanted to use this to motivate the Even-Paz Protocol. Search it up on Wikipedia!

Doing this upto a precision of ϵ needs $O(n \log(n) \log(\epsilon^{-1}))$ time. Can you see how?

Merge Shuffle

Problem. Shuffle an array using as few calls to a random generator as possible.

The random generator produces a number uniformly at random from $[0, 1]$.

Goal: Design an algorithm to produce a uniformly random permutation of the array with the minimum expected number of random calls.

Solution

The idea I know of is very similar to merge sort. This idea was initally published by Heinrich Apfelmus on Valentine's Day 2009. It was then rediscovered indipendently by a team at University of Paris and Princeton in 2015. And I had rediscovered it again in my second semester making BlackJack in Haskell.

Idea is very similar to a riffle shuffle if you know what that means. The only difference is that after a riffle shuffle the deck is not random... unless the halves were random to begin with.

Although, there is one more bump in the road. If we choose a card from the two piles using a coin flip; once the piles get uneven, the permutation will no longer be random and be more weighted towards permutations where prefix embedding the smaller array is shorter. (Read this twice and run an example on paper to convince yourself) So what do we do?

Look at the pseudo code

1. Split the list into two lists L and R
2. Let $l = \text{len}(L)$ and $r = \text{len}(R)$
3. **while** $L \neq \emptyset$ and $R \neq \emptyset$
   1. Generate random number x in $[0, 1]$
   2. **if** $x < \frac{l}{l+r}$
      - Choose element from L and append to output
      - Remove element from L
      - $l = l - 1$
   3. **else**
      - Choose element from R and append to output
      - Remove element from R
      - $r = r - 1$
4. Append remaining elements of the non-empty list

It is easy to see we use $R(n) = 2R(\frac{R}{2}) + n \implies R(n) = O(n \log(n))$ random calls.

The Word "Algorithm" — A Brief Etymology

The word **algorithm** originates from French, where it was the mistranslated name of the 9th-century Arabic scholar **Al-Khwarizmi**.

Born in present-day Uzbekistan, he studied and worked in Baghdad. His text on multiplying Indo-Arabic numerals travelled to Europe, and his name was mistranslated as *Algorisme* — which later evolved into *algorithm*.

The multiplication algorithm we all learned in school? **That's his.**

Naive Multiplication

The naive approach: define multiplication as **repeated addition**.

$$\texttt{multiply}(1, b) = b, \qquad \texttt{multiply}(a, b) = b + \texttt{multiply}(a - 1, b)$$

For two n-digit numbers this takes:

$$\mathcal{O}(10^n) \text{ additions} \times \mathcal{O}(n) \text{ per addition} = \mathcal{O}(n \cdot 10^n)$$

(Fine for single-digit multiplication where we'd otherwise need to write out a times table by hand. Utterly useless for large numbers.)

Schoolbook Multiplication (Al-Khwarizmi, 9th Century)

The familiar grade-school algorithm: multiply each digit of one number against every digit of the other, shift, and add.

For 32×45: compute 32×5 and 32×4, then add with appropriate shift.

Complexity: $\mathcal{O}(n^2)$ — we multiply all n digits of one number with all n digits of the other, then add n results of length $\leq 2n$.

This was the state of the art for over **a thousand years**.

Naive Divide & Conquer

Split: $x = a \cdot 10^{n/2} + b$, $y = c \cdot 10^{n/2} + d$. Then:

$$xy = ac \cdot 10^n + bc \cdot 10^{n/2} + ad \cdot 10^{n/2} + bd$$

This gives **4 recursive subproblems** of size $n/2$, plus $\mathcal{O}(n)$ additions.

Recurrence:

$$T(n) = 4T(n/2) + \mathcal{O}(n)$$

By the Master Theorem: $T(n) = \mathcal{O}(n^{\log_2 4}) = \mathcal{O}(n^2)$.

No improvement over schoolbook! All that effort for nothing . . . or is it?

Karatsuba's Insight (1962)

Key observation. We only need three terms:

$$xy = ac \cdot 10^n + \underbrace{(bc + ad)}_{\text{need this}} \cdot 10^{n/2} + bd$$

Genius: We already compute ac and bd. Notice:

$$(a + b)(c + d) = ac + bd + (bc + ad)$$

So: $bc + ad = (a + b)(c + d) - ac - bd$.

Only **3 recursive multiplications** needed! (Note: $a + b$ and $c + d$ are at most $n/2 + 1$ digits.)

New recurrence:

$$T(n) = 3T(n/2) + \mathcal{O}(n) \implies T(n) = \mathcal{O}(n^{\log_2 3}) \approx \mathcal{O}(n^{1.585})$$

Karatsuba: Pseudocode

```
def karatsuba(x, y):
    # x, y: digit lists, least significant first
    if len(x) == 1: return mul_by_digit(y, x[0])
    if len(y) == 1: return mul_by_digit(x, y[0])

    n2 = max(len(x), len(y)) // 2
    a, b = x[n2:], x[:n2]    # x = a * 10^n2 + b
    c, d = y[n2:], y[:n2]    # y = c * 10^n2 + d

    ac         = karatsuba(a, c)
    bd         = karatsuba(b, d)
    abcd       = karatsuba(add(a, b), add(c, d))
    ad_plus_bc = sub(sub(abcd, ac), bd)

    # xy = ac*10^(2*n2) + (ad+bc)*10^n2 + bd
    return add(add(shift(ac, 2*n2),
                   shift(ad_plus_bc, n2)), bd)
```

Beyond Karatsuba: Toom-Cook (1963)

Just one year after Karatsuba, Toom and Cook generalised the idea.

Key generalisation: Split each number into k parts of size n/k instead of 2 parts. With the right interpolation, only $2k-1$ recursive multiplications are needed.

k	Name	Complexity
2	Toom-2 = Karatsuba	$\mathcal{O}(n^{\log_2 3}) \approx \mathcal{O}(n^{1.585})$
3	Toom-3	$\mathcal{O}(n^{\log_3 5}) \approx \mathcal{O}(n^{1.465})$
k	Toom-k	$\mathcal{O}(n^{\log_k(2k-1)})$

As $k \to \infty$, the exponent $\log_k(2k-1) \to 1$. But the hidden constants grow **exponentially** with k, making large k impractical.

Exercise: Implement Toom-3 by modifying Karatsuba.

The Race to $\mathcal{O}(n \log n)$

Year	Authors	Complexity
9th C	Al-Khwarizmi (schoolbook)	$\mathcal{O}(n^2)$
1962	Karatsuba	$\mathcal{O}(n^{1.585})$
1963	Toom-Cook	$\mathcal{O}(n^{\log_3 5})$
1971	Schönhage–Strassen	$\mathcal{O}(n \log n \log \log n)$
2007	Fürer (Using Complex Analysis)	$\mathcal{O}(n \log n \cdot 2^{\mathcal{O}(\log^* n)})$
2008	De, Kurur, Saha, Saptharishi† (Using p-adic fields)	$\mathcal{O}(n \log n \cdot 2^{\mathcal{O}(\log^* n)})$
2019	Harvey–Hoeven	$\mathcal{O}(n \log n)$

† Saptharishi was at CMI when this was done; others were from IIT Kanpur.

Harvey and Hoeven on their 2019 result: *"...our work is expected to be the end of the road for this problem, although we don't know yet how to prove this rigorously."*

They shared the **de Bruijn Medal** in 2022.

What's Actually Used in Practice?

The algorithms from 2007 onward are **galactic algorithms** — they beat prior methods only for astronomically large inputs (e.g., Fürer beats Schönhage–Strassen only for integers with about 10^{19} digits).

In practice, even the most sophisticated mathematical software uses:

- **Karatsuba** for moderate sizes
- **Toom-3** for larger sizes
- **Schönhage–Strassen** (via FFT over integers) for very large sizes

Schönhage–Strassen (and everything onwards) uses something called the **Discrete Fast Fourier Transform**. Do ask me if you want to know more about it. It is very classic and very pretty divide and conquer algorithm.